

Aforro — Backend Developer Assignment (Round-2)

Overview

Build a small but complete backend module that demonstrates your understanding of Django, REST API design, data modeling, query optimization, asynchronous processing, caching, and containerized development.

This assignment is practical and reflects the type of systems we build at Aforro.

Quality, clarity, and correctness matter more than size.

1. Functional Requirements

1.1 Data Models

Implement the following models:

Category

- name

Product

- title
- description (optional)
- price
- category (FK to Category)

Store

- name
- location

Inventory

- store
- product
- quantity
- Note: A store must have at most one inventory row per product.

Order

- store
- status: PENDING | CONFIRMED | REJECTED
- created_at

OrderItem

- order
- product
- quantity_requested

1.2 Order Creation

Endpoint: POST /orders/

Input contains:

- store_id
- list of product_id + quantity_requested items

Rules:

1. Validate product availability for the requested store.
 2. If any product has insufficient stock:
 - Order must be created with REJECTED status
 - No stock should be deducted
 3. If all items have sufficient stock:
 - Deduct quantities
 - Mark order as CONFIRMED
 4. The entire operation must be wrapped inside transaction.atomic() to guarantee consistency.
 5. Return a JSON response with final status and order details.
-

1.3 Order Listing

Endpoint: GET /stores/<store_id>/orders/

Return a list of all orders belonging to the store:

- order ID
- status
- created_at
- total number of items

Sort by newest first and ensure efficient queries (no N+1 issues).

1.4 Inventory Listing

Endpoint: GET /stores/<store_id>/inventory/

Return inventory items for that store including:

- product title
- price
- category name
- quantity

Sort results alphabetically by product title.

1.5 Product Search API

Endpoint: GET /api/search/products/

Requirements:

- Keyword search on product title, description, and category name
- Optional filters:
 - category
 - price range
 - store_id
 - in_stock

- Sorting:
 - price
 - newest
 - relevance
- Include pagination metadata
- If store_id is provided, include the product's inventory quantity for that store

Full-text search or multi-field contains logic is acceptable.

1.6 Autocomplete API

Endpoint: GET /api/search/suggest/?q=xxx

Rules:

- Minimum 3 characters required
 - Return up to 10 product titles
 - Prefix matches should appear before general matches
 - Keep response size small and the logic fast
-

1.7 Dummy Data Generator

Provide a Django management command:

```
python manage.py seed_data
```

It should generate:

- 10+ categories
- 1000+ products
- 20+ stores
- Inventory for each store with at least 300 products

Use Faker or any seed data generator of your choice.

2. Engineering Requirements

2.1 Redis Integration

Use Redis in one of the following ways (your choice):

Option A — Caching

Implement caching for one API (for example: product search, autocomplete, or inventory listing).

Ensure proper cache invalidation.

Option B — Rate Limiting

Apply request rate limiting to:

GET /api/search/suggest/

Suggested limit: 20 requests per minute per user/IP.

2.2 Celery Integration

Set up Celery using Redis as the message broker.

Add one asynchronous job such as:

- Sending order confirmation

- Generating daily inventory summaries
- Preprocessing products for search

Document how the worker runs and how tasks are triggered.

2.3 Docker Setup

Containerize the project using Docker.

A docker-compose.yml should start:

- Django API server
 - PostgreSQL
 - Redis
 - Celery worker
 - (Optional) Celery beat
-

3. Project Structure

Recommended structure:

```
project/  
  apps/  
    products/  
    stores/  
    orders/  
    search/  
  tests/  
  settings.py  
  requirements.txt  
  README.md
```

4. Submission

Your GitHub repository must include:

- Complete Django project
- All required models, APIs, serializers, URLs
- Seed data command
- Redis (cache or rate limit) implementation
- Celery task integration
- Docker environment
- At least **3–5 tests**
- README containing:
 - Setup instructions
 - Docker usage
 - Sample API requests
 - Notes on caching/async logic
 - Scalability considerations

5. Next Round

If selected, the next session will include:

- Walkthrough of your search and autocomplete logic

- Order/inventory handling
- Database consistency reasoning
- Redis caching or rate-limit logic
- Celery workflow
- How you structured and reasoned about the project
- Discussion on improvements and trade-offs